Still Needed, Hard to Transfer: Using and Reusing Contextual Skills

Anonymous Author(s)

Abstract

Agent skills provide task-specific capabilities by loading instructions and supporting resources into a language model's context. We investigate what allows the internal states produced from a skill to support another execution. Across model depth, we transplant correct-skill residual states, the hidden vectors passed between model blocks, into a wrong-skill execution. Separately, we block subsequent attention to the skill in the original execution. Across three models from two families on MedCalc and TheoremQA, residual-state transfer weakens before dependence on source access subsides. Content, position and cross-question controls establish that early transfer carries structured skill information. To investigate the later separation, we jointly replace skill states and the source positions' key/value (KV) cache, which stores attention vectors for previously processed tokens. On Qwen3-8B/MedCalc, this raises layer-16 accuracy from 3.5% with residual replacement alone to 42.1%. Cutting subsequent reading of the repaired source reduces accuracy to 7.0%. Skill-state transfer thus depends on more than the replaced residual vectors: compatible source caches can restore part of the effect, while continued source access can remain necessary for the recovered behaviour. The mechanism evidence covers one model–task setting and identifies source caches and subsequent reading as concrete factors to test when reusing contextual skills.

CCS Concepts

• **Computing methodologies** → **Natural language processing**; *Knowledge representation and reasoning*.

Keywords

agent skills, Web agents, large language models, in-context learning, activation patching, context reuse, mechanistic interpretability

1 Introduction

Relevance to the Web and to this track. Web agents rely on machine-readable instructions and descriptions of tools and services to understand how to act across sites [22, 23]. Agent Skills package such procedural knowledge for reuse. We ask how a model uses a loaded skill and when the internal states it produces can support another execution. This connects our study to the Semantics and Knowledge track's interest in machine-readable descriptions that help agents understand and use Web resources, and in knowledge representations that support intelligent agent behaviour.

An Agent Skill packages task instructions, scripts and supporting resources around a `SKILL.md` entry point, which an agent loads when needed [26]. We study the instruction content of a loaded skill. A model can apply that content successfully, but how much of the resulting ability can be reused through its internal states? Reusing states from an earlier execution is an appealing way to share skills and avoid repeated processing. It requires understanding which states contribute to the ability and what else those states need in order to work.

A useful starting point is to move skill states into an execution using an unrelated skill and measure the benefit restored. Task and function vectors show that internal states can transmit task behaviour across contexts [10, 20]. Such transfer may also depend on the computation surrounding the states and on further access to the source. We therefore ask: *how does the ability to transfer a skill's states relate to the ongoing need to read that skill?*

We follow both properties across model depth. First, we replace the hidden vectors at skill-token positions in a wrong-skill execution with vectors from a correct-skill execution. These residual states are the vectors passed between successive model blocks. Second, we block attention to the skill from a chosen layer through the rest of the original correct-skill execution. The fraction of originally correct answers lost measures source-attention dependence.

Figure 1 previews the result. Early residual-state replacement recovers much of the skill's benefit. Deeper in the model, transfer becomes weak while blocking subsequent skill attention still damages the original execution. This ordering recurs across three models from two families and two task domains, including comparisons on identical eligible questions. We use content, position and cross-question controls to establish the effective early probe before interpreting this separation.

What can restore transfer in that interval? Residual replacement leaves the receiver's attention cache in place. This key/value (KV) cache stores vectors that later tokens read through attention. Jointly replacing the skill's residual states and its source-position cache entries restores substantial late transfer in Qwen3-8B/MedCalc. Cutting subsequent reading then removes most of the recovered effect. The source cache and continued reading therefore matter alongside the residual vectors themselves. Earlier reader computation remains unchanged in these tests, so the intervention targets specific source components rather than replaying the donor's complete execution.

Work on layer-wise computation and reusable skill caches highlights how depth and stored context shape model behaviour [13, 25]. We connect these questions through three contributions. First, we validate an early skill-span transfer probe using content, position and cross-question controls (§4.2). Second, we identify a depth interval in which residual transfer is weak while the original execution still depends on skill attention (§4.3). Third, we isolate the contributions of source-position caches and subsequent reading to late transfer in one model–task setting (§4.4). Together, the results identify concrete conditions under which skill states can be reused: the accompanying source cache and the access available after transfer both deserve explicit tests.

WWW '27, Dublin, Ireland
2027.

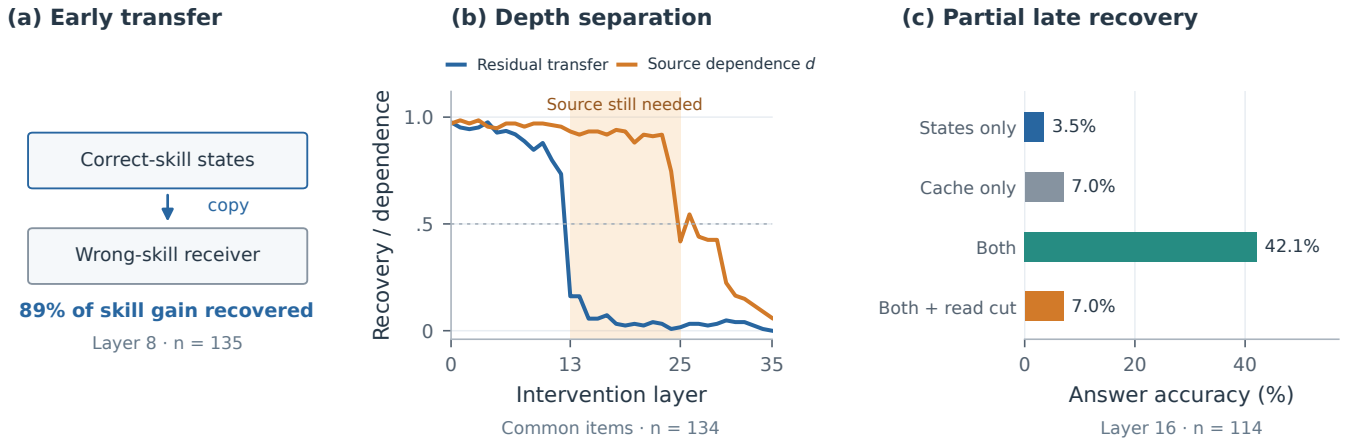


Figure 1: From effective early transfer to source-cache repair. (a) Early skill states recover most of the donor's benefit on the screened Qwen3-8B/MedCalc set. (b) On common eligible items, residual transfer weakens while source dependence remains high. Dependence is $d(L) = 1 - q(L)$, the fraction of rescued answers lost after cutting source access. (c) At layer 16, replacing source states together with their source cache recovers part of late transfer, and cutting subsequent reading removes most of that recovery. Panels use their indicated evaluation populations.

2 Related Work

Transferring task behaviour through internal states. Task vectors summarize demonstrations as internal vectors that can steer a new example [10]. Function vectors identify attention-head outputs that transmit a task's behaviour across contexts [20]. Natural-language instructions also yield transferable components [5], and instruction patches can interact across layers [2]. The effect of transfer varies with the task and intervention time [15]. These results establish useful transfer settings. We examine how transfer changes across depth relative to the original execution's continued dependence on its skill source.

Tracing context use and execution history. Cumulative context masking tests whether a model still needs source tokens at later layers [18]. Instruction-span and final-position patches can reveal different effects [17]. Joint analyses of masking and state transfer locate subtask execution across layers [25]. Activation patching, the replacement of selected hidden states by states from another run, requires matched controls and careful implementation [9, 27]. Effects can depend on interactions among the intervened components [21]. We pair full-depth transfer and masking profiles, then test source states together with their attention caches. This connects our work to PrefillNotes [13], which shows that precompiled skill caches can be repositioned and composed with a target context. Our focus is when residual-state reuse weakens while source access remains important, and how replacing source-position caches affects that separation.

Skills and reusable context. Agent Skills organize instructions and resources for on-demand use [26]. Benchmarks measure skill retrieval and application [8, 14, 19], while matched-budget evaluations show that their benefit depends on the available computation [7]. We fix the selected skill and study how its loaded content supports execution. A related direction compresses context into aggregated activations [1], learned summary tokens [16], compact hidden states [6], or weight updates [3, 4]. We study naturally occurring states through interventions, providing tests of reuse and source dependence that complement trained compression methods.

3 Problem Formulation

Skill context and intervention questions. Our unit of analysis is a skill's instruction body once loaded into model context. We supply this content directly, holding skill selection fixed, and isolate how the model uses and reuses it internally.

State transfer tests recovery in a wrong-skill receiver. Attention knockout tests source dependence in the correct-skill run. Both score generated answers while varying intervention depth. The model first processes the prompt, called prefill, and then generates answer tokens, called decoding. We call tokens outside the skill span readers, including question and answer tokens. Figure 2 summarizes which source components are replaced and which attention paths are cut.

Skill-span state transfer. We call the loaded instruction text the skill source and its token positions the skill span. Residual-stream states are the hidden vectors passed from one transformer block to the next. Let x be a question, s its correct skill and $\tilde{s}$ an unrelated, equal-length skill. For hidden width D, let H^L_{skill} and H^L_{wrong} denote residual-stream states at layer L under these two contexts. Across the skill's m token positions S, the content-specific difference is a matrix $d^L_S = H^L_{\text{skill}}(S) - H^L_{\text{wrong}}(S) \in \mathbb{R}^{m\times D}$. We use a run containing the wrong skill as the *receiver*, and the correct-skill run as the *donor*. Injection at dose α sets

$$H^L_{\text{patched}}(S) = H^L_{\text{wrong}}(S) + \alpha d^L_S. \tag{1}$$

At $\alpha = 1$, the donor's states replace the receiver's across the span. Other states remain unchanged at that layer. Computation then continues. Matching lengths preserves positions and their encodings.

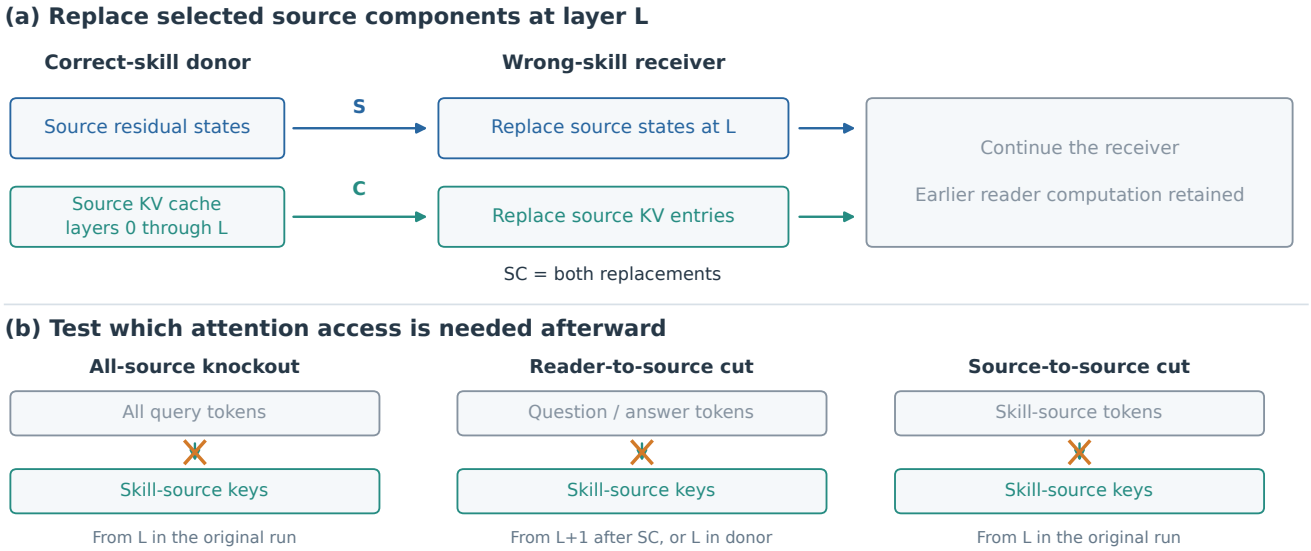


Figure 2: Intervention design. (a) The correct-skill donor supplies source residual states (S), source-position KV entries through layer L (C), or both (SC) to the wrong-skill receiver. Earlier reader computation is retained. (b) Attention cuts distinguish access by reader tokens from updates within the skill source. After SC, the reader cut begins at $L+1$. In the original correct-skill run, path cuts begin at L. All cuts continue through the remaining prompt processing and decoding.

Span interventions change many more states than final-position interventions, which limits direct comparisons. Here the final position is the prompt's end, after the query. State patches are applied during prefill, after which decoding proceeds without further patches. The intervention replaces a block's output, not the complete execution state. Attention uses a query vector to select key vectors and read their associated value vectors. Key/value caches store these vectors for previously processed tokens. At the patched block and earlier blocks, these caches retain their receiver values. Later generated tokens can still read them. This distinction motivates the joint state-and-cache intervention in §4.4.

Behavioural recovery and evaluation subsets. We score generated answers and measure the fraction of the donor's accuracy gain recovered by an intervention. Higher recovery means more of the correct skill's benefit has been restored:

$$\rho(\text{arm}) = \frac{\text{acc}(\text{arm}) - \text{acc}(\text{receiver})}{\text{acc}(\text{donor}) - \text{acc}(\text{receiver})}. \tag{2}$$

Here, an arm is an intervention condition, and the donor baseline uses the correct skill in context. Thus $\rho = 1$ recovers the donor's observed gain, and values above one indicate greater accuracy than the donor. We call questions answered incorrectly without a skill and correctly with it *rescued items*. Main causal comparisons use these items. For span interventions, we further retain only skill groups for which the wrong-skill receiver solves none of the sampled items. We apply this screen separately within each run, model and task, without using intervention outcomes. The estimates are conditional on baseline failure in the sampled group, rather than recovery over the full benchmark.

Measuring source-attention dependence. Cumulative attention knockout masks the skill positions as attention keys at every layer from L onward, during both prefill and decoding. On the knockout run's rescued items within the screened groups, let $q(L)$ be the fraction still solved after this intervention. We define *source-attention dependence* as

$$d(L) = 1 - q(L). \tag{3}$$

Because these items are correct in the unmodified execution, $d(L)$ is the fraction that become incorrect under the knockout. Higher values indicate stronger behavioural dependence on source attention from layer L onward. The mask applies to skill keys for all query positions, including skill queries during prefill. It therefore jointly intervenes on reading and source-state updates, rather than isolating a reader-to-source pathway. The transfer boundary is the first layer with $\rho(L) < 0.5$ after early successful transfer. The source-dependence boundary is the first with $d(L) \leq 0.5$, equivalently $q(L) \geq 0.5$. Layers with $\rho(L) < 0.5$ and $d(L) > 0.5$ exhibit weak residual-state transfer despite strong source-attention dependence. Shading marks the interval between the first half-effect boundaries. We report later recrossings directly in the curves. We also repeat both curves on the intersection of transfer items and the knockout run's rescued items within screened groups, retaining each intervention's own baselines. The curves describe the effects of the two interventions in their respective computational backgrounds.

4 Experiments

4.1 Experimental setup

Tasks and models. We pair MedCalc-Bench [12] with skills from SRA-Bench [19], covering 1,100 questions from 55 clinical calculators. We also use SRA-Bench's TheoremQA subset, with 747 questions and 320 skills. Each question receives its designated skill, holding skill selection fixed. We test Qwen3-8B, Qwen3-0.6B [24]

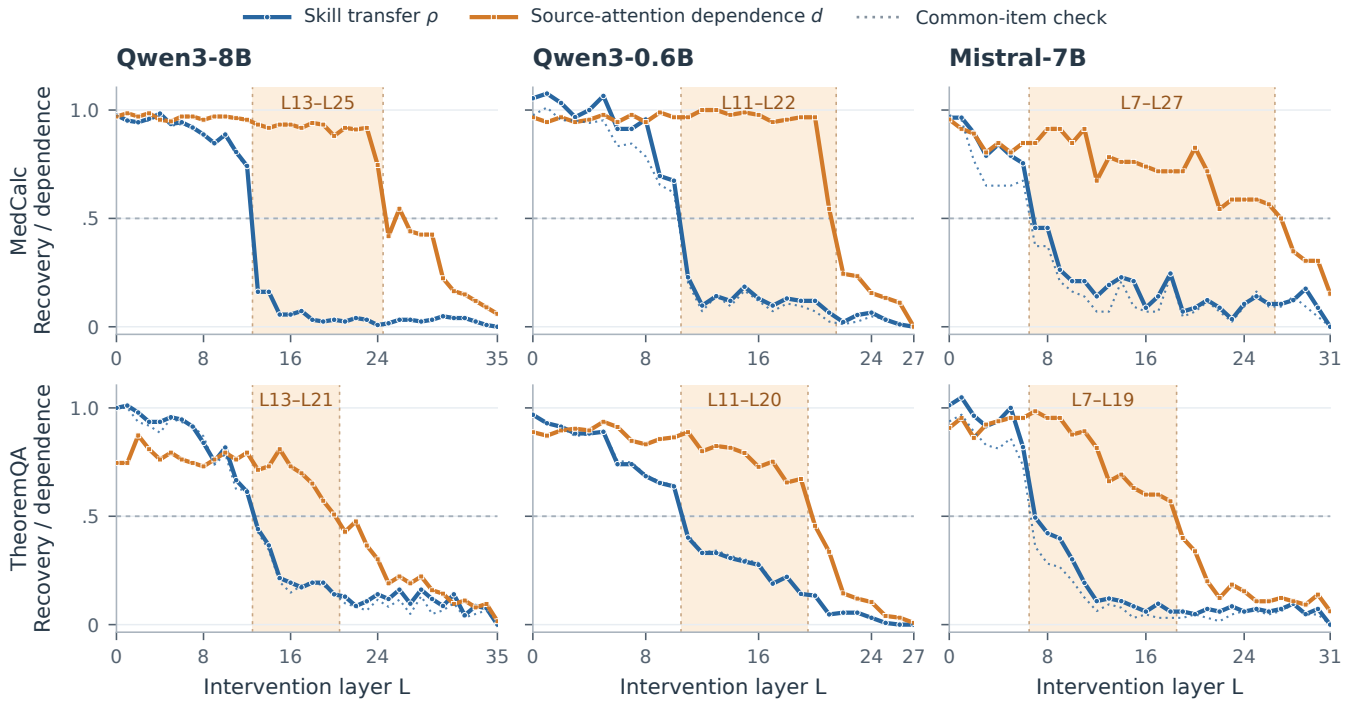


Figure 3: Depth profiles across three models and two tasks. Blue shows skill-span transfer recovery $\rho(L)$ in a wrong-skill receiver. Orange shows source-attention dependence $d(L) = 1 - q(L)$: the fraction of originally correct answers lost after cumulative attention knockout in the correct-skill execution. Dotted traces repeat the tests on common eligible questions. Shading spans the first half-effect boundaries, highlighting weak transfer despite strong dependence; later recrossings remain visible. Boundary layers are marked above each panel. Every plotted layer is measured.

and Mistral-7B-Instruct-v0.3 [11], spanning two model families. Models generate step-by-step answers with greedy decoding, selecting the highest-probability next token at each step. We use no tools and apply the corresponding benchmark scorers. On the full MedCalc benchmark, Qwen3-8B achieves 74.2% accuracy with the correct skill, 34.5% without a skill, and 31.5% with an unrelated skill.

Evaluation populations. Our causal comparisons focus on questions rescued by the correct skill, as defined in §3. On MedCalc/Qwen3-8B, 467 rescued questions from 48 calculators enter the span-intervention battery. Retaining skill groups whose wrong-skill receiver solves no sampled item gives 135 questions from 14 calculators. The depth sweeps use this group screen, with separately eligible transfer and knockout items. We also recompute both curves on their common item IDs. TheoremQA uses the same screen, becoming item-level for skills with one question. The source-cache replacement study uses a separate fixed set described in §4.4. These populations estimate effects conditional on baseline skill benefit.

Implementation and uncertainty. State replacement occurs during prefill, the processing of prompt tokens before answer generation. Cumulative attention knockout applies during both prefill and decoding. An identity patch writes the receiver's own states back and should reproduce its output. Correcting an attention-mask mismatch between state capture and decoding restores exact agreement, including all 467 MedCalc identity checks. All main results use the corrected implementation. Appendix A.1 specifies the intervention protocol, and Appendix A.2 records the correction. We estimate 95% confidence intervals by resampling whole skill groups with replacement, preserving dependence among questions using the same skill. This skill-group bootstrap measures uncertainty across the sampled groups rather than treating their questions as independent.

4.2 Validating a probe of reusable skill content

Early span replacement recovers the skill's contribution. Before comparing transfer across depth, we establish that the intervention transmits useful behaviour. At layer 8 on the screened MedCalc set, replacing states at the skill's token positions recovers most of the correct skill's benefit (Table 1). Patching the last prompt position or the final 16 positions has a much smaller effect. This locates an effective intervention in the skill span. Its larger number of replaced states also makes it a higher-capacity intervention than the final-position alternatives.

The effect depends on skill content and position. The unrelated-skill, permuted-position and noise controls in Table 1 all yield little recovery. The noise is scaled to match the size, measured by Euclidean norm, of the skill-state change. For the content comparison,

Table 1: Early transfer controls, Qwen3-8B/MedCalc, layer 8: 135 items from 14 calculators. Accuracy and recovery ρ are point estimates. Final-position patches replace fewer states than the skill-span patches.

Intervention	Accuracy	ρ
Correct skill donor	.919	1.00
Wrong skill receiver	.000	.00
Correct skill span	.815	.89
Last prompt position	.030	.03
Last 16 prompt positions	.059	.06
Other skill content	.044	.05
Correct span, permuted	.044	.05
Norm-matched noise	.022	.02
Identity patch	.000	.00

the correct and unrelated skills are patched over a common prefix length. The correct prefix recovers 0.65 of the available gain. A mean of same-family skill differences recovers 0.12 on its smaller eligible set of 41 items. Rescaling patched states to match the receiver's norm at each position preserves 0.87 recovery, close to the full-span result. These comparisons identify skill-specific content and its arrangement as important to early transfer. The group screen matters: other-skill and noise controls are more effective in excluded groups, so the control contrast describes the screened population.

Early transfer generalizes across questions using the same skill. In the main prompt order, the skill precedes the question. Causal attention, which restricts a token to attending to earlier positions, prevents those skill states from encoding the later question. We additionally place the skill after the question and obtain donor states from other questions using the same skill. On 151 eligible items from 17 calculators, these states recover 0.98 at layer 8, close to 1.02 for a donor answering the current question. At layer 16, the values separate to 0.29 and 0.49. Early transfer therefore carries information shared across instances, with greater question sensitivity later. Patching individual contiguous quarters of the main skill span also reveals content structure: the formula quarter gives the largest standalone recovery, 0.56. Appendix B provides the additional content and prompt-order controls.

4.3 Residual-state transfer weakens before source dependence subsides

The two depth profiles separate. With an effective early probe established, we follow state transfer and source-attention knockout across every layer. Figure 3 shows the six model–task combinations. For Qwen3-8B/MedCalc, transfer recovery is 0.74 at layer 12, falls to 0.16 at 13, and is at most 0.07 from layer 15 onward. For cuts beginning at $L \leq 23$ and continuing through the final layer during prompt processing and answer generation, source-attention dependence is 0.88–0.97: most originally correct answers are lost. Dependence is 0.75 at layer 24 and first falls to at most one half at layer 25. Throughout layers 13–24, transfer recovers less than half the gain, while removing further skill attention still loses more than half the rescued answers.

Table 2: Boundary layers on common eligible items. T/R gives the first half-effect crossings for transfer ($\rho < 0.5$) and source dependence ($d \leq 0.5$), respectively. $\Delta = R - T$ measures their separation in layers. The last column gives its skill-group bootstrap 95% confidence interval.

Task	Model	n	T/R	Δ	95% CI
MedCalc	Qwen3-8B	134	13/25	12	[12.0, 17.0]
MedCalc	Qwen3-0.6B	90	11/22	11	[10.0, 13.0]
MedCalc	Mistral-7B	46	7/27	20	[10.0, 24.0]
TheoremQA	Qwen3-8B	63	13/21	8	[5.0, 10.0]
TheoremQA	Qwen3-0.6B	125	11/20	9	[8.0, 10.0]
TheoremQA	Mistral-7B	65	7/19	12	[9.0, 13.0]

The first crossing summarizes each curve. Qwen3-8B/MedCalc dependence briefly rises above half again at layer 26 before falling below it from layer 27 onward. We retain these fluctuations in the measured curves. The main finding is the separation of the two intervention effects: states become ineffective at correcting the receiving computation while source access still supports the original one.

The ordering holds on common questions across settings. Original transfer and knockout runs can retain different eligible questions. We therefore recompute the curves over the intersection of their item IDs, using eligibility determined from unpatched baselines. Table 2 reports the resulting boundaries. All six pairs match those from the original populations. Skill-group bootstrap 95% intervals for the source-dependence-minus-transfer boundary difference exclude zero in all six settings. Using transfer thresholds $\tau = 0.4$ or 0.6 and the corresponding dependence thresholds $1 - \tau$ also preserves the ordering. The gap thus recurs across the two model families and both task domains under common-item evaluation.

Repeated late replacement leaves transfer weak. A single write might be insufficient to sustain a transferred skill. We test this by replacing source-span states over consecutive layer windows on the same 135 MedCalc items. Windows 8–11, 12–15, 14–19 and 16–35 recover 0.89, 0.74, 0.16 and 0.06, respectively. Replacing states at every layer from the start recovers 0.98. Repetition is therefore effective when begun early, while starting at layer 16 gives the same weak recovery as a single write there. A corresponding TheoremQA test also yields weak late-window recovery (Appendix B). This directs attention to the execution source-position cache entries that residual replacement leaves unchanged.

4.4 Testing source caches and continued reading

Replacing source states and source-position caches. A residual-state patch changes a block output while leaving previously computed source-cache entries in place. To test whether these entries matter, we compare source residual replacement (S), source key/value-cache replacement (C), their joint replacement (SC), and SC followed by cutting reader-to-source attention from the next layer. The cache condition replaces source entries through the intervention layer, keeping earlier reader computation fixed.

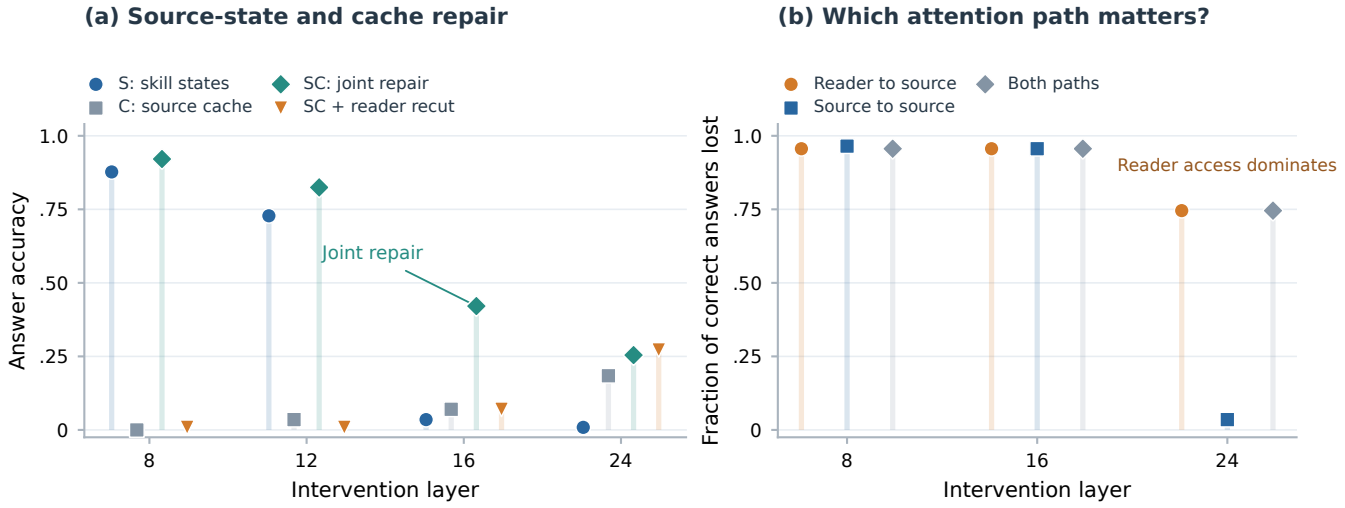


Figure 4: Source-state/cache replacement and attention-path tests on the same 114 Qwen3-8B/MedCalc items from 14 skills. (a) Accuracy following residual replacement (S), cache replacement (C), joint repair (SC), or joint repair followed by a reader cut from the next layer. (b) Fraction of donor-correct answers lost when reader-to-source, source-to-source, or both attention paths are cut from the indicated layer onward. Points are measured estimates, horizontally offset within each layer for legibility. Table 3 reports the main contrasts.

Table 3: Focused mechanism contrasts, 114 items from 14 skills. Differences are in percentage points (pp). The additive interaction is $SC - S - C$ + receiver.

Contrast	Difference
L16: $SC - S$	+38.6
L16: additive interaction	+31.6
L16: $SC - (SC + \text{recut})$	+35.1
L24: reader minus self loss	+71.1

We evaluate layers 8, 12, 16, 24 on a fixed 124-question set from 14 calculators with fresh baselines. Of these, 114 questions are correct with the donor and incorrect with both the wrong-skill and no-skill baselines. Both panels of Figure 4 use these same questions. All 48 own-state controls across four GPU shards reproduce output tokens exactly. Appendix B.1 gives implementation and eligibility details.

Joint repair restores substantial late transfer. At layer 16, residual replacement answers 4/114 questions correctly, cache replacement answers 8/114, and joint repair answers 48/114. Table 3 reports the repair gain. The joint gain exceeds the sum of the individual gains relative to the receiver, showing that the states and their source caches contribute together. At layer 24, joint repair still improves on residual replacement (25.4% versus 0.9%), although its advantage over cache replacement alone (18.4%) is uncertain. Compatible source caches thus recover part of the late transfer effect.

Recovered transfer still uses subsequent source reading. Following joint repair at layer 16, cutting reader-to-source attention reduces accuracy to 8/114. Most of the repaired effect therefore relies on continued reading. To identify what the original cumulative knockout disrupts, we separately cut reader-to-source attention, source-to-source attention, or both in the unmodified donor, during prefill and decoding. At layer 24, reader-only and combined cuts each leave 29/114 questions correct, while a source-self-only cut leaves 110/114 correct. Continued reader access accounts for the dominant tested dependence at this layer. Earlier cuts at layers 8 and 16 damage both paths.

The repair effect varies with depth. After joint repair at layer 24, a reader recut yields 27.2% accuracy, with no detected additional loss. The repaired-yet-reading-dependent result is therefore supported at layer 16 and varies across the tested depths. The main repair gain is stable to generation-length diagnostics: it remains 38.5 points on the 96 questions whose three baselines finish without truncation. An adverse reassignment of truncated outputs still leaves a 35.1-point gain (Appendix B.1). Taken together, the experiments show that replacing source-position caches can restore part of late residual transfer, and that the recovered behaviour can still depend on reading the skill.

5 Conclusion

Skill-state transfer is shaped by both the transferred component and the source access available afterward. Across three models and two task domains, residual-state transfer weakens before dependence on skill attention subsides. Content and cross-question controls establish that the effective early intervention carries structured skill information.

The focused Qwen3-8B/MedCalc study makes this separation more concrete. Replacing source-position KV caches together with residual states restores substantial late transfer at layer 16, while

cutting subsequent source reading removes most of the recovered effect. This identifies source caches and continued reading as two contributors to successful reuse. The intervention leaves earlier reader computation unchanged, and the recutting effect varies with depth, so these results characterize the tested source components rather than all of the receiver's computational history.

For systems that reuse contextual skills, residual-state recovery, source-cache compatibility and subsequent source access are therefore distinct properties to evaluate. Extending the joint interventions to additional models and interactive agent tasks can establish how broadly these conditions govern skill reuse.

References

[1] Thibaud Ardoin, Semira Einsele, Evis Bregu, and Gerhard Wunder. 2026. Prompt Compression via Activation Aggregation. *arXiv preprint arXiv:2607.08399* (2026).

[2] Irina Bigoulaeva, Jonas Rohweder, Subhabrata Dutta, and Iryna Gurevych. 2026. Patches of Nonlinearity: Instruction Vectors in Large Language Models. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, San Diego, California, United States, 12209–12262. doi:10.18653/v1/2026.acl-long.559

[3] Lucas Caccia, Alan Ansell, Edoardo Ponti, Ivan Vulić, and Alessandro Sordoni. 2025. Training Plug-n-Play Knowledge Modules with Deep Context Distillation. In *Conference on Language Modeling (COLM)*. arXiv:2503.08727.

[4] Rujikorn Charakorn, Edoardo Cetin, Shinnosuke Uesaka, and Robert Tjarko Lange. 2026. Doc-to-LoRA: Learning to Instantly Internalize Contexts. *arXiv preprint arXiv:2602.15902* (2026).

[5] Guy Davidson, Todd M. Gureckis, Brenden M. Lake, and Adina Williams. 2025. Do different prompting methods yield a common task representation in language models? *arXiv preprint arXiv:2505.12075* (2025). https://arxiv.org/abs/2505.12075

[6] Tao Ge, Jing Hu, Lei Wang, Xun Wang, Si-Qing Chen, and Furu Wei. 2024. In-context Autoencoder for Context Compression in a Large Language Model. In *International Conference on Learning Representations (ICLR)*. arXiv:2307.06945.

[7] Sina Hajimiri, Masih Aminbeidokhti, Jose Dolz, Ismail Ben Ayed, Issam H. Laradji, Spandana Gella, and Nicolas Gontier. 2026. Are Online Skill and Memory Modules Always Worth Their Tokens? A Budget-Constrained Study of Web Agents. *arXiv preprint arXiv:2606.15017* (2026). https://arxiv.org/abs/2606.15017

[8] Tingxu Han, Yi Zhang, Wei Song, Chunrong Fang, Zhenyu Chen, Youcheng Sun, and Lijie Hu. 2026. SWE-Skills-Bench: Do Agent Skills Actually Help in Real-World Software Engineering? *arXiv preprint arXiv:2603.15401* (2026).

[9] Stefan Heimersheim and Neel Nanda. 2024. How to use and interpret activation patching. *arXiv preprint arXiv:2404.15255* (2024).

[10] Roee Hendel, Mor Geva, and Amir Globerson. 2023. In-Context Learning Creates Task Vectors. In *Findings of the Association for Computational Linguistics: EMNLP 2023*. arXiv:2310.15916.

[11] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, et al. 2023. Mistral 7B. *arXiv preprint arXiv:2310.06825* (2023).

[12] Nikhil Khandekar, Qiao Jin, Guangzhi Xiong, Soren Dunn, Serina S. Applebaum, Zain Anwar, Maame Sarfo-Gyamfi, Conrad W. Safranek, Abid A. Anwar, Andrew Zhang, Aidan Gilson, Maxwell B. Singer, Amisha Dave, Andrew Taylor, Aidong Zhang, Qingyu Chen, and Zhiyong Lu. 2024. MedCalc-Bench: Evaluating Large Language Models for Medical Calculations. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*. arXiv:2406.12036.

[13] Bojie Li. 2026. Models Take Notes at Prefill: KV Cache Can Be Editable and Composable. *arXiv preprint arXiv:2606.17107* (2026). https://arxiv.org/abs/2606.17107

[14] Xiangyi Li, Yimin Liu, Wenbo Chen, et al. 2026. SkillsBench: Benchmarking How Well Agent Skills Work Across Diverse Tasks. *arXiv preprint arXiv:2602.12670* (2026).

[15] Yuxuan Li, Declan Campbell, Stephanie C. Y. Chan, and Andrew Kyle Lampinen. 2025. Just-in-time and distributed task representations in language models. *arXiv preprint arXiv:2509.04466* (2025).

[16] Jesse Mu, Xiang Lisa Li, and Noah Goodman. 2023. Learning to Compress Prompts with Gist Tokens. In *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:2304.08467.

[17] Aditya Pola and Vineeth N. Balasubramanian. 2025. Where does an LLM begin computing an instruction?. In *UniReps Workshop*. extended abstract; arXiv:2511.10694.

[18] Suzanna Sia, David Mueller, and Kevin Duh. 2024. Where does In-context Learning Happen in Large Language Models?. In *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:2403.04510.

[19] Weihang Su, Jianming Long, Qingyao Ai, Qiaozhi He, Yichen Tang, Changyue Wang, Yiteng Tu, Yingbo Wang, and Yiqun Liu. 2026. Skill Retrieval Augmentation for Agentic AI. *arXiv preprint arXiv:2604.24594* (2026).

[20] Eric Todd, Millicent L. Li, Arnab Sen Sharma, Aaron Mueller, Byron C. Wallace, and David Bau. 2024. Function Vectors in Large Language Models. In *International Conference on Learning Representations (ICLR)*. arXiv:2310.15213.

[21] Sankaran Vaidyanathan, David Arbour, Aaron Mueller, Scott Niekum, and David Jensen. 2026. The Curse of Multiple Mediators: Hidden Interaction Effects in Activation Patching. *arXiv preprint arXiv:2606.27510* (2026). https://arxiv.org/abs/2606.27510

[22] Zhaoyang Wang, Qianhui Wu, Xuchao Zhang, Chaoyun Zhang, Wenlin Yao, Fazle Elahi Faisal, Baolin Peng, Si Qin, Suman Nath, Qingwei Lin, Chetan Bansal, Dongmei Zhang, Saravan Rajmohan, Jianfeng Gao, and Huaxiu Yao. 2026. WebXSkill: Skill Learning for Autonomous Web Agents. *arXiv preprint arXiv:2604.13318* (2026). https://arxiv.org/abs/2604.13318

[23] Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. 2025. Agent Workflow Memory. In *Proceedings of the 42nd International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 267)*. PMLR, 63897–63911. https://proceedings.mlr.press/v267/wang25bx.html

[24] An Yang, Anfeng Li, et al. 2025. Qwen3 Technical Report. *arXiv preprint arXiv:2505.09388* (2025).

[25] Zhipeng Yang, Junzhuo Li, Siyu Xia, and Xuming Hu. 2025. Internal Chain-of-Thought: Empirical Evidence for Layer-wise Subtask Scheduling in LLMs. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. 22536–22564. doi:10.18653/v1/2025.emnlp-main.1147

[26] Barry Zhang, Keith Lazuka, and Mahesh Murag. 2025. Equipping agents for the real world with Agent Skills. Anthropic Engineering. https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills

[27] Fred Zhang and Neel Nanda. 2024. Towards Best Practices of Activation Patching in Language Models: Metrics and Methods. In *International Conference on Learning Representations (ICLR)*. arXiv:2309.16042.

A Experimental protocols and provenance

Tasks and scoring. The main MedCalc evaluation uses the 55 calculator skills released with SRA-Bench and 20 questions per calculator. The benchmark scorer checks free-form numerical answers against its prescribed tolerances. TheoremQA uses its released prompts, skill assignments and evaluator. All main experiments supply the selected skill directly and generate answers without tool calls. MedCalc questions share calculator definitions, so uncertainty estimates resample calculator groups. For TheoremQA, groups are the corresponding skill identities.

Evaluation populations. We distinguish four baseline outcomes: rescued items are incorrect without the skill and correct with it, persistent failures are incorrect in both conditions, kept items are correct in both, and broken items become incorrect with the skill. The main interventions focus on rescued items. The additional receiver screen retains skill groups in which the wrong-skill receiver solves no sampled question. This reduces recovery through disruption of an interfering wrong skill, concentrating the comparison on the contribution of the donor's content. Appendix B compares this group screen with a less restrictive screen that excludes only individually solved receiver items.

A.1 Implementation details

Models and decoding. The main models are Qwen3-8B (36 layers, hidden width 4096), Qwen3-0.6B (28 layers, width 1024), and Mistral-7B-Instruct-v0.3 (32 layers, width 4096). For Mistral, which has no system role in its chat template, system instructions are merged into the user turn. We use greedy decoding. Qwen thinking mode is disabled and checked on the rendered prompt. Attention-mask interventions use explicit attention computation with mask checks, often called eager attention. Earlier residual-replacement runs use PyTorch scaled dot-product attention. The mechanism

Table 4: Full-dataset rerun against the development subset, Qwen3-8B on MedCalc, layer 8. Recovery is Eq. 2; entries show point estimates. *per-calc* is the main text's restriction, *per-item* the weaker one. Controls reach 0.12 under the group screen and 0.25 under the item screen; each arm uses its own matched baselines.

arm	full, per-calc (n = 135)	subset (n = 60)	full, per-item (n = 369)
transplant `real`	**0.89**	0.87	0.94
same skill, matched prefix	0.65	0.73	0.84
half dose $\alpha = 0.5$	0.04	0.06	0.11
positions permuted	0.05	0.08	0.21
norm-matched noise	0.02	0.10	0.18
other calculator	0.05	—	0.25
other family, mean	0.05	0.06	0.16
same family, mean	0.12	0.05	0.24
identity patch	0.00	0.00	0.00
rank 128 / 64 / 32 / 16	0.77/0.63/0.12/0.05	0.77/0.54/0.13/0.06	0.88/0.70/0.22/0.18
bottom-64 / bottom-16	0.03/0.04	0.00/0.06	0.11/0.12
quarters $q_0/q_1/q_2/q_3$	0.09/**0.56**/0.12/0.05	0.12/0.56/0.12/0.12	0.18/0.64/0.22/0.19

experiments use BF16, a 16-bit floating-point format, and a 1,536-token generation limit.

Position and state checks. Donor and receiver prompts use length-matched skills, preserving question positions. Residual patches replace the selected block output during prefill. The captured positions are indexed within the prompt, separately from any answer tokens used for scoring. Identity controls write the receiver's own states back. A final-layer check also verifies that replacing a position's final hidden state reproduces the donor's next-token scores at that position. Attention cuts preserve the causal mask.

Focused mechanism protocol. A bridge on one fixed item from each of 14 calculators compares full-prompt prefill with prefill split before the final prompt token. All 84 own-state checks reproduce their corresponding receiver token sequences exactly. Changing the prefill protocol changes one donor's and one receiver's answer correctness. The formal repair and path comparisons therefore use the same split protocol and fresh no-skill, donor and receiver baselines. Truncation counts and sensitivity analyses are retained with the result records.

A.2 Evaluation scope and intervention provenance

Attention-mask correction. The original state-capture call omitted the attention mask supplied during decoding. Although mathematically equivalent for an unpadded sequence, these calls selected different numerical kernels. Their states differed by 1.2% at layer 8 and 25% at layer 34. Identity patching changed 108/1,766 Qwen3-8B outcomes (6.1%). Matching the masks restores exact identity agreement, including all 467 items in the corrected MedCalc rerun. All main-text state interventions use this correction. Attention knockout operates directly on attention masks and uses no donor-state capture.

Development and replication. The initial MedCalc intervention battery sampled four rescued questions from each of the 40 calculators with the most eligible questions. The corrected full-data rerun uses every rescued question from calculators with at least two, giving 467 items from 48 calculators. The main receiver screen then retains 135 questions from 14 calculators. Historical development results retain their original sample labels. Table 4 shows the corrected rerun alongside the earlier screened subset. Main depth comparisons cover all six model–task combinations, with common-item checks for their two intervention types.

Scientific scope. The experiments characterize supplied skill instructions under fixed selection and controlled generation. The depth ordering is replicated across the tested models and tasks. The source-history repair and path-specific tests provide a more focused result on Qwen3-8B/MedCalc. Interactive browser tasks, skill retrieval errors and learned state compression are natural settings in which to test how these findings extend to agent systems.

B Additional transfer and source-reading analyses

The original MedCalc × Qwen3-8B intervention battery was developed on the forty calculators with the most rescued items, sampling four each: 160 of 467 rescued items, and 60 items over 15 calculators after the receiver-never-solves restriction. We reran each of them with every rescued item of every calculator that has at least two (467 items, 48 calculators), keeping the intervention configuration fixed. The group screen is stricter on the larger set because a calculator must fail on up to twenty items rather than four, so it keeps 135 items over 14 calculators, and we also report the per-item restriction (369 items over 45), which keeps the items whose own receiver fails.

Table 4 also includes a rank-truncation diagnostic. We decompose the injected state-difference matrix into ordered directions using singular value decomposition and retain only the leading k. Increasing k from 32 to 64 restores much of the recovery. A bottom-k control retains the least prominent directions instead. The formula quarter gives the largest standalone recovery in both samples. The same-family mean donor changes from 0.05 to 0.12, on the 41 full-rerun items where it is available.

The depth sweep at one-layer resolution. The subset sweep measured layers 0, 4, 8, 12, 14, 15, 16, 20 and placed the collapse somewhere after 12. At full scale we added 13 and 17–19: recovery is 0.98, 0.98, 0.89, 0.74 at layers 0, 4, 8, 12, then 0.16, 0.16, 0.06, 0.06, 0.07, 0.03, 0.02, 0.03 at 13 through 20 (per-calc; the per-item restriction gives 1.00, 0.98, 0.94, 0.79 then 0.32, 0.34, 0.25, 0.25, 0.23, 0.19, 0.14, 0.17). Under the group screen, layers 13 and 14 both round to 0.16. Under the item screen, they differ at 0.32 and 0.34. Both screens show their largest local decline between layers 12 and 13.

Repeated replacement on TheoremQA.. The corrected Qwen3-8B comparison uses 99 eligible questions from 75 skills with matched donor and wrong-skill receiver baselines. Replacing source residual states at each layer of windows 8–11, 12–15, 14–19 and 16–35 recovers 0.839, 0.613, 0.376 and 0.204, respectively. Layer 16 alone also recovers 0.204. These values come from the completed `t8-win` and `tqa-span` runs, whose compatible baselines are merged before applying the receiver screen. As in MedCalc, extending replacement from layer 16 through the final layer adds little recovery over the single-layer intervention.

A continuous readout at the same boundary. Accuracy thresholds a generated answer, so an internal change can appear sharper in this measure. We therefore generate the correct-skill reasoning once per item and score that fixed sequence under each intervention. This is teacher forcing: the model receives the recorded preceding tokens rather than generating a new sequence. The mean token log-probability measures how strongly it predicts that reasoning. Log-probabilities use natural logarithms, with units called nats. Higher values indicate that the recorded sequence is more likely.

We normalize this score as $z = (\ell_L - \ell_{\text{recv}})/(\ell_{\text{gold}} - \ell_{\text{recv}})$, where ℓ denotes the mean log-probability and the subscripts identify the patched, receiver and donor conditions. We use z to distinguish this diagnostic from knockout retention q. On 442 donor-correct items from 48 calculators, z is 0.999, 0.989, 0.960, 0.831 at layers 0, 4, 8, 12. It then falls to 0.598 (95% interval [0.550, 0.648]) at layer 13, 0.595 at 14, 0.340 at 16, 0.201 at 20, and 0.129 at 24. The layer-12 to layer-13 decline spans 26.8% of the score's range, compared with the 61% behavioural decline. The continuous readout therefore also changes sharply at the accuracy boundary.

The decline is distributed across the four quarters of the reasoning sequence (0.20, 0.26, 0.31, 0.21). Scoring only tokens after the final answer marker is less useful here. Once the reasoning is supplied, all conditions predict the final number closely: median log-probabilities lie between −0.001 and −0.005, and the donor-minus-receiver denominator has median 0.0014, below 0.01 on 59% of items.

Distributed reading and receiver state. An attention head is one of the parallel attention components within a transformer block. Head-level readout from the skill span concentrates in layers 12–21 after normalisation by residual-stream norm. Blocking the 32 highest-ranked heads costs 0.39 of the skill's effect, versus 0.23 for random heads at the same layers (held-out calculators; paired difference 0.154, 95% CI [0.061, 0.251]). Reading is distributed rather than confined to a small circuit. After late span replacement, downstream readout remains 0.86–0.99 of the donor's despite weak behavioural recovery. Earlier receiver question-state updates remain unchanged, motivating the interaction tests in §4.4.

B.1 Focused source-history and path-specific experiments

The focused Qwen3-8B/MedCalc experiments fix 124 historical item IDs from 14 calculators before inference and partition them as every fourth ID across four RTX 4090 workers. They use BF16, eager attention, greedy decoding and 1,536 new tokens. Fresh baselines retain 116 donor-correct, receiver-incorrect items, of which 114 also fail without a skill. Both experiments use these same 114 items; all 372 baseline generations agree token-for-token between the two suites. The complete repair grid contains 2,356 generations, the path grid 1,488, and worker identity checks 100. All 48 own-state writebacks are token-exact.

S writes the donor source-span block output at layer L. C replaces only source-position keys and values at layers 0 through L after prefix prefill and before recomputing the final prompt token. SC combines them; SC recutting blocks reader queries to source keys from $L + 1$ onward in that same run. Earlier reader history is unchanged. Reader queries are positions after the source span; source-self queries are positions within it. The path experiments cut the specified edges during both prefill and decoding, preserving the causal mask.

We retain all intervention outcomes, including truncated outputs. For the prespecified SC-minus-S contrast at layer 16, the gain is 38.6 percentage points, with a skill-cluster bootstrap 95% interval of [26.6, 46.5] (10,000 resamples of the 14 calculators, retaining their items). The corresponding joint-minus-additive contrast is 31.6 points [14.1, 41.6]; SC-minus-recut is 35.1 points [23.2, 43.0]. On the 96 items with all three baselines untruncated, SC-minus-S is 38.5 points [25.0, 47.6]. At layer 16, S/SC/recut truncate 4/4/3 of 114 outputs; worst-case reassignment of all truncated S and SC outcomes leaves a 35.1-point repair advantage. At layer 24, SC and recut truncate 3/28 outputs and do not exhibit a reliable recutting loss; this depth is not used to claim independence after repair.

At layer 24, reader and source-self cuts retain 29/114 and 110/114 correct answers. Their loss difference is 71.1 points [53.3, 86.5]. Reader cuts truncate 21 outputs; counting all as correct still leaves retention at 50/114, below source-self retention 110/114 (no truncation). Both contrasts survive worst-case reassignment of truncated outputs. Analysis artifacts retain arm-wise counts, hashes, clustered intervals and truncation bounds.